

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÁO CÁO BTL Phát triển ứng dụng trên thiết bị di động

Tinner: Nền tảng di động tối ưu hóa quá trình đặt món và địa điểm

Giảng viên: Ths. Hoàng Lê Hải Thanh
Lớp: L01
Khoa: Khoa học và Kỹ thuật Máy tính

TP. HỒ CHÍ MINH, THÁNG 05/2026

Danh sách thành viên và Phân công

Dưới đây là danh sách các thành viên tham gia thực hiện bài tập lớn và bảng phân công chi tiết:

MSSV	Thành viên	Phân công
2313063	Phạm Ngọc Tây	[Nhiệm vụ chi tiết]
[MSSV]	Âu Nguyễn Hùng Mạnh	[Nhiệm vụ chi tiết]
[MSSV]	Nguyễn Tiến Nam	[Nhiệm vụ chi tiết]
[MSSV]	Phan Đình Khang	[Nhiệm vụ chi tiết]
[MSSV]	Thanh Nhân	[Nhiệm vụ chi tiết]

Bảng 1: Bảng phân công nhóm

Mục lục

Danh sách thành viên và Phân công	1
1 Giới thiệu	4
1.1 Đặt vấn đề	4
1.2 Mục tiêu và Mục đích	4
1.3 Nhu cầu thực tế và Tác động tiềm năng	5
2 Tiếp cận và Phương pháp luận	5
2.1 Quy trình phát triển	5
2.2 Công cụ cộng tác	5
2.3 Cơ sở cho việc ra quyết định	5
3 Tính năng và Chức năng của ứng dụng	6
3.1 Các tính năng cốt lõi	6
3.2 Các chức năng nâng cao	6
4 Chi tiết triển khai	7
4.1 Tổ chức mã nguồn	7
4.2 Backend triển khai tại <code>apps/backend</code>	7
4.2.1 Luồng khởi động và middleware chung	7
4.2.2 Xác thực và phân quyền	8
4.2.3 Các module nghiệp vụ	8
4.2.4 Tầng gọi ý và tích hợp Overpass	8
4.2.5 Health check	9
4.3 Ứng dụng di động triển khai tại <code>apps/mobile</code>	9
4.4 Giao diện web tham chiếu tại <code>frontend</code>	9
4.5 Phát triển cục bộ và kiểm thử	10
4.5.1 Chạy hệ thống cục bộ bằng Docker Compose	10
4.5.2 Quy trình migration	10
4.5.3 Kiểm thử backend	10
5 Trải nghiệm người dùng	10
5.1 Luồng người dùng chi tiết	10
5.1.1 Tính năng 1: Vuốt để khám phá và lưu món ăn	10
5.1.2 Tính năng 2: Quản lý danh sách “Muốn thử”	11
5.2 Ảnh chụp màn hình	11
6 Sơ đồ kiến trúc hệ thống	14
6.1 Mô tả kiến trúc tổng quan	14
6.2 Sơ đồ	14
6.3 Luồng yêu cầu gọi ý nhà hàng	15
7 Thiết kế dịch vụ web	15
7.1 Kiểu dịch vụ và quy ước định tuyến	15
7.2 Xác thực và Phân quyền	16
7.3 Validation, Xử lý lỗi và Bảo mật	16
7.4 Tài liệu hóa API	17

8	Chiến lược triển khai	17
8.1	Mục tiêu triển khai	17
8.2	Công nghệ triển khai	18
8.3	Triển khai Backend trên Render	18
8.4	Chiến lược cơ sở dữ liệu trên Aiven	19
8.5	Triển khai Frontend trên Vercel	19
8.6	Phát hành ứng dụng di động qua EAS	19
8.7	CI/CD và Vận hành	19
8.8	Tóm tắt	20
9	Báo cáo độ bao phủ kiểm thử	20
9.1	Giới thiệu Jest và các công cụ testing	20
9.2	Cấu hình môi trường kiểm thử	20
9.3	Tiêu chuẩn độ bao phủ	21
9.4	Kết quả kiểm thử chi tiết	21
9.4.1	Module Auth (Xác thực)	22
9.4.2	Module Gợi ý món ăn	22
9.4.3	Module Tương tác người dùng	22
9.4.4	Module Bộ lọc và Món ăn	22
9.5	Các kịch bản kiểm thử API	23
9.6	Phạm vi kiểm thử E2E Layer	23
9.6.1	Phương pháp tiếp cận và Cấu trúc kiểm thử E2E	24
9.6.2	Đánh giá kết quả kiểm thử E2E	24
9.7	Phân tích khoảng trống kiểm thử và Chất lượng mã nguồn	25
9.7.1	Phân tích khoảng trống kiểm thử (Gap Analysis)	25
9.7.2	Đề xuất cải thiện chất lượng mã nguồn dựa trên kiểm thử	26
10	Phân tích khảo sát mức độ hài lòng và Hành vi người dùng	27
10.1	Phản hồi và Kết quả khảo sát người dùng	27
10.2	Số liệu hành vi người dùng (Analytics)	27
11	Đánh giá dự án và Các hướng cải thiện	29
11.1	Thành công và Thách thức	29
11.1.1	Thành công đạt được (Achievements)	29
11.1.2	Thách thức đối mặt (Challenges)	29
11.1.3	Bài học kinh nghiệm (Lessons Learned)	30
11.2	Các kế hoạch trong tương lai	30
12	Sản phẩm thực tế	31
12.1	Khả năng tương thích của ứng dụng	31
12.2	Tệp APK và Liên kết Google Drive	31
12.3	Tích hợp Backend và Cấu hình Môi trường Bảo mật	32
13	Phân phối sản phẩm và Khảo sát	34
13.1	Quảng bá và Tiếp thị	34
13.2	Thiết kế khảo sát và Thu thập số liệu ứng dụng	34
13.3	Đánh giá phản hồi và Phân tích hành vi	35
13.4	Minh chứng tương tác người dùng	35

1 Giới thiệu

1.1 Đặt vấn đề

Trong cuộc sống hiện đại, đặc biệt là đối với sinh viên, nhân viên văn phòng và các cặp đôi, câu hỏi "Hôm nay ăn gì?" đã phát triển từ một câu hỏi đơn giản thành một nguồn gây căng thẳng hàng ngày. Vấn đề này biểu hiện qua ba điểm yếu (pain points) chính:

- **Sự mệt mỏi khi ra quyết định (Decision Fatigue):** Trung bình, mỗi cá nhân dành từ 15 đến 20 phút để lướt các ứng dụng giao đồ ăn một cách vô định mà không đưa ra được lựa chọn cuối cùng, thường dẫn đến việc chọn một bữa ăn ngẫu nhiên, không hài lòng hoặc bỏ bữa hoàn toàn.
- **Quá tải lựa chọn (Choice Overload):** Các nền tảng hiện nay hiển thị hàng trăm tùy chọn nhà hàng cùng lúc dưới dạng danh sách. Lượng thông tin khổng lồ này gây ra tình trạng quá tải nhận thức, khiến người dùng khó đưa ra quyết định và ít hài lòng hơn với lựa chọn cuối cùng của mình.
- **Mâu thuẫn nhóm (Social Friction):** Việc quyết định địa điểm ăn uống là một nguồn gây mâu thuẫn phổ biến cho các nhóm, thường dẫn đến sự bực bội hoặc những thỏa hiệp không thoải mái.

1.2 Mục tiêu và Mục đích

Mục tiêu chính của dự án này là phát triển **Tinner** (một từ ghép của "Tinder" và "Dinner"), một ứng dụng di động được thiết kế để giải quyết vấn đề "Tê liệt phân tích" (Analysis Paralysis) trong việc lựa chọn thực phẩm. Với khẩu hiệu "*Swipe To Bite*", Tinner đơn giản hóa quy trình ra quyết định.

Các mục đích cốt lõi của ứng dụng bao gồm:

- **Hợp lý hóa các quyết định:** Giúp người dùng chuyển từ việc mở ứng dụng sang quyết định một bữa ăn trong vòng chưa đầy 5 phút.
- **Trải nghiệm ưu tiên hình ảnh (Visual-First Experience):** Cung cấp một "Giao diện chế độ xem đơn" (Single-View Interface) hiển thị hình ảnh món ăn chất lượng cao tại một thời điểm, loại bỏ các yếu tố gây xao nhãng và kích thích sự thèm ăn.
- **Khám phá thông qua Gamification:** Triển khai cơ chế quẹt thẻ (Vuốt sang trái để bỏ qua, Vuốt sang phải để lưu/thích) giúp biến một công việc tẻ nhạt thành một hoạt động hấp dẫn.
- **Truy cập thông tin liền mạch:** Khi thích một món ăn, ứng dụng ngay lập tức hiển thị Thẻ thông tin nhà hàng với các chi tiết quan trọng (địa chỉ, giờ mở cửa, tích hợp bản đồ và đánh giá) để hoàn tất quyết định.
- **Quản lý hành động:** Cho phép người dùng duy trì danh sách "Muốn thử" cá nhân để xem xét, so sánh và điều hướng đến các nhà hàng đã chọn một cách hiệu quả.

1.3 Nhu cầu thực tế và Tác động tiềm năng

Nghiên cứu thị trường toàn diện chỉ ra một khoảng trống đáng kể trong hệ sinh thái ứng dụng liên quan đến thực phẩm hiện nay. Trong khi các siêu ứng dụng giao hàng vượt trội về hậu cần, các nền tảng mạng xã hội vượt trội về cảm hứng hình ảnh và dịch vụ bản đồ vượt trội về dữ liệu vị trí, thì không có mô hình lai (hybrid model) ưu việt nào được tối ưu hóa thuần túy cho việc khám phá nhà hàng nhanh chóng, thú vị và có cấu trúc.

Tinner giải quyết nhu cầu thực tế này bằng cách định vị mình là một "Công cụ ra quyết định" (Decision Engine) hơn là một trung gian giao dịch. Bằng cách tập trung vào giai đoạn khám phá và kết hợp các tính năng như Bộ lọc thông minh (Smart Filters) và Bộ sưu tập cá nhân (Personal Collections), ứng dụng giúp giảm tải nhận thức và thu hẹp khoảng cách giữa thông tin có cấu trúc và trải nghiệm người dùng hấp dẫn.

Tác động tiềm năng của Tinner là rất lớn, đặc biệt là đối với Gen Z và Millennials. Nó hứa hẹn sẽ giảm đáng kể thời gian lãng phí cho các quyết định ăn uống hàng ngày, giảm bớt mâu thuẫn xã hội liên quan đến lựa chọn của nhóm và cung cấp một giải pháp trực quan và thú vị cho một vấn đề nan giải hàng ngày.

2 Tiếp cận và Phương pháp luận

2.1 Quy trình phát triển

Để đảm bảo cung cấp nhanh chóng Sản phẩm khả thi tối thiểu (Minimum Viable Product - MVP) và cho phép cải tiến lặp đi lặp lại, dự án đã áp dụng phương pháp luận phát triển Agile. Cách tiếp cận này cho phép nhóm ưu tiên các tính năng cốt lõi—cụ thể là các chức năng khám phá và ra quyết định—trong khi vẫn duy trì tính linh hoạt để kết hợp phản hồi của người dùng. Chu kỳ phát triển được chia thành các giai đoạn (sprints) tập trung, cho phép thử nghiệm liên tục, tinh chỉnh giao diện người dùng và xác nhận triết lý thiết kế "Ưu tiên hình ảnh" (Visual-First).

2.2 Công cụ cộng tác

Việc giao tiếp hiệu quả và kiểm soát phiên bản (Version Control) được duy trì bằng cách sử dụng các công cụ cộng tác tiêu chuẩn công nghiệp. Git và GitHub được sử dụng để quản lý mã nguồn, cho phép nhóm xử lý việc phát triển tính năng song song, thực hiện đánh giá mã (Code Review) và theo dõi các vấn đề (Issues) một cách liền mạch. Các tài liệu như luồng người dùng (User Flows), nghiên cứu thị trường và ma trận tính năng được duy trì theo định dạng Markdown trong kho lưu trữ để đảm bảo tất cả các thành viên trong nhóm đều có quyền truy cập vào một nguồn thông tin duy nhất (Single Source of Truth).

2.3 Cơ sở cho việc ra quyết định

Việc lựa chọn tính năng và kiến trúc thiết kế được thúc đẩy bởi mục tiêu giải quyết các điểm yếu (pain points) cụ thể của người dùng, như đã nêu trong Cơ sở lý luận sản phẩm (Product Rationale):

- **Tập trung vào các giá trị cốt lõi (Khám phá & Quyết định):** Ưu tiên không phải là xây dựng một danh bạ tĩnh, mà là một công cụ động trực tiếp chấm dứt sự thiếu quyết đoán của người dùng. Các tính năng được đánh giá dựa trên Ma trận

Tác động và Nỗ lực (Impact vs. Effort Matrix), ưu tiên các chức năng có tác động cao như Thẻ thông tin (Info Card) và Bộ sưu tập cá nhân (Personal Collections).

- **Triết lý "Ưu tiên hình ảnh" (Visual-First):** Hoạt động trên nguyên tắc "người ta ăn bằng mắt", thiết kế ưu tiên mạnh mẽ hình ảnh thực phẩm rực rỡ hơn là các menu dày đặc văn bản. Điều này tạo niềm tin và khơi gợi cảm xúc ngay từ cái nhìn đầu tiên.
- **Giải quyết nghịch lý của sự lựa chọn (Paradox of Choice):** Bằng cách chỉ hiển thị một món ăn tại một thời điểm và sử dụng cơ chế quẹt thẻ (Gamified swipe mechanism), ứng dụng cố tình giới hạn các tùy chọn đồng thời để giảm tải nhận thức và biến việc ra quyết định thành một trải nghiệm giải trí.
- **Chiến lược duy trì người dùng (Retention Strategy):** Danh sách "Muốn thử" (Bộ sưu tập cá nhân) được triển khai như một "lưỡi câu" chính (primary hook). Khi người dùng bỏ công sức để lưu các địa điểm yêu thích của họ, họ tạo ra một cơ sở dữ liệu cá nhân có giá trị, làm tăng đáng kể Tỷ lệ duy trì (Retention Rate) của ứng dụng.

3 Tính năng và Chức năng của ứng dụng

3.1 Các tính năng cốt lõi

Sản phẩm khả thi tối thiểu (MVP) tập trung vào việc cung cấp luồng người dùng thiết yếu: khám phá các món ăn, lưu các sở thích, xem lại các lựa chọn và điều hướng đến điểm đến.

- **Vuốt để khám phá (Cơ chế Swipe-to-Match):** Đây là cơ chế khám phá chính. Hệ thống lấy các món ăn gần đó dựa trên vị trí của người dùng và hiển thị chúng trong Giao diện Chế độ xem Đơn (Single-View Interface). Người dùng vuốt sang trái để bỏ qua hoặc vuốt sang phải để lưu món ăn. Cơ chế này thúc đẩy việc ra quyết định nhanh chóng và trực quan.
- **Quản lý danh sách "Muốn thử" (Bộ sưu tập cá nhân):** Một không gian dành riêng nơi người dùng có thể xem lại và quản lý các món ăn họ đã lưu trước đó thông qua việc vuốt. Nó cho phép người dùng so sánh các lựa chọn, cân nhắc lại các lựa chọn và xóa các mục, chuyển đổi giai đoạn khám phá thành một quy trình ra quyết định có thể hành động.
- **Thẻ thông tin nhà hàng (Restaurant Info Card):** Khi tương tác với một món ăn, ứng dụng sẽ hiển thị một thẻ toàn diện bao gồm hình ảnh thực phẩm chất lượng cao, tóm tắt thực đơn, giờ mở cửa và các đánh giá nổi bật. Điều này cung cấp một cái nhìn tổng quan nhanh chóng mà không yêu cầu chuyển đổi nhiều màn hình.

3.2 Các chức năng nâng cao

Để nâng cao trải nghiệm người dùng ngoài việc khám phá cơ bản, một số chức năng nâng cao đã được tích hợp vào nền tảng:

- **Bộ lọc thông minh (Smart Filters):** Người dùng có thể thu hẹp kết quả tìm kiếm một cách chính xác dựa trên nhu cầu thời gian thực bằng cách lọc theo bán kính (khoảng cách), mức giá, loại ẩm thực và xếp hạng.
- **Tích hợp Bản đồ & Điều hướng:** Người dùng có thể xem các nhà hàng đã thích trên bản đồ hoặc điều hướng trực tiếp đến vị trí bằng một lần chạm thông qua việc tích hợp Google Maps hoặc Apple Maps, giúp tối ưu hóa lộ trình di chuyển.
- **Tích hợp API bên thứ ba:** Để tăng uy tín dữ liệu và cung cấp cái nhìn khách quan, ứng dụng tích hợp với các nguồn bên ngoài (ví dụ: Google Places, Yelp) để hiển thị điểm số và đánh giá đã được xác minh.
- **Khớp nhóm và Gamification (Group Match and Gamification):** Các tính năng như "Khớp nhóm" (khớp chỉ xảy ra khi mọi người trong nhóm cùng vượt sang phải) và "Vòng quay may mắn" được thiết kế để giải quyết các xung đột ăn uống trong nhóm và biến việc chọn nhà hàng thành một trò chơi xã hội thú vị.
- **Tour ẩm thực (Food Tours):** Ứng dụng hỗ trợ các hành trình được tuyển chọn để biến một bữa tối tiêu chuẩn thành một cuộc phiêu lưu nhỏ, kích thích sự tò mò và tăng thời gian người dùng tương tác với nền tảng.

4 Chi tiết triển khai

4.1 Tổ chức mã nguồn

Dự án tổ chức theo mô hình monorepo, sử dụng `pnpm` làm trình quản lý gói/workspace, yêu cầu Node.js 22+. Các thành phần chính:

- **apps/backend:** Backend REST API xây dựng bằng NestJS 11, dùng Prisma 7 + PostgreSQL.
- **apps/mobile:** Ứng dụng di động Expo (React Native + TypeScript). Phần native chính nằm trong `apps/mobile/app-native` (entry: `AppShell.tsx`, các screen `Login/Signup/Swipe/Map/Collections/Filters`).
- **packages/types:** DTO/enum dùng chung giữa backend và client.
- **frontend:** Giao diện web tham chiếu (Vite + React), không bắt buộc cho luồng mobile chính.

4.2 Backend triển khai tại apps/backend

Backend dùng TypeScript với framework NestJS 11. Truy cập dữ liệu qua Prisma 7 kết nối PostgreSQL (Aiven trên production, Docker Postgres trên local).

4.2.1 Luồng khởi động và middleware chung

Trong `main.ts`, ứng dụng bật CORS với `credentials`, dùng `cookie-parser`, đăng ký:

- Một global filter `GlobalExceptionHandler (@Catch())` bắt mọi exception — phân biệt `HttpException` (trả status/message đã chuẩn hoá) và các lỗi không lường trước (log full server-side, trả 500 `Internal server error`). Ở môi trường non-production, response thêm trường `details` chứa danh sách lỗi validation; ở production thì giữ message chung để không lộ chi tiết.
- Một global `ValidationPipe` với `transform`, `whitelist`, `forbidNonWhitelisted`.
- Swagger UI tại `/api/docs` (xem mục 7.4).

4.2.2 Xác thực và phân quyền

Hệ thống dùng JWT (`@nestjs/jwt`, `passport-jwt`) với 2 token:

- `accessToken` (TTL 15 phút, mặc định) — gửi trong header `Authorization: Bearer`.
- `refreshToken` (TTL 30 ngày, mặc định) — lưu hash trong bảng `refresh_tokens`, dùng để rotate access token.

`JwtAuthGuard` đăng ký toàn cục qua `APP_GUARD` trong `AppModule`; các endpoint công khai đánh dấu bằng decorator `@Public()` (ví dụ `/auth/register`, `/auth/login`, `/auth/refresh`, `/health`).

Luồng đăng ký (POST `/api/v1/auth/register`) yêu cầu `username` (3–50 ký tự, chữ/số/underscore), `email`, `password` (8–128, có chữ và số), `confirmPassword` và `fullName` tùy chọn. Sau khi tạo user, hệ thống đồng thời khởi tạo bản ghi `user_filters` mặc định để các luồng *suggestions* sẵn sàng dùng ngay.

4.2.3 Các module nghiệp vụ

- `AuthModule`: `register/login/refresh/logout/me`.
- `DishesModule`: trả danh sách món ăn (16 dish enum: phở, bún bò, cơm tấm, bánh mì...).
- `FiltersModule`: lưu `maxDistanceKm`, `minRating`, `cuisines`, `dietary`, `priceRanges` cho từng user.
- `InteractionsModule`: ghi nhận hành vi swipe-right (`SAVE_RESTAURANT`) và like-dish (`LIKE_DISH`).
- `SuggestionsModule`: cốt lõi của trải nghiệm swipe — chi tiết ở 4.2.4.
- `HealthController`: endpoint kiểm tra trạng thái (xem 4.2.5).

4.2.4 Tăng gợi ý và tích hợp Overpass

`SuggestionsService` dùng chiến lược *DB-first* kết hợp Overpass API (OpenStreetMap) làm nguồn dữ liệu:

1. Với mỗi request GET `/api/v1/suggestions?lat&lng[&dishType]`, service đếm số nhà hàng đã đồng bộ gần đây trong bounding box quanh user (cột `lastSyncedAt` ≤ 7 ngày). Nếu đạt ngưỡng (mặc định 20 quán) thì dùng luôn dữ liệu DB.

2. Khi vùng chưa fresh, gọi 1 query Overpass API trong bán kính 7 km, filter amenity in {restaurant, fast_food, cafe, food_court}, sau đó bulk insert qua `prisma.restaurant.createMany({ skipDuplicates: true })`.
3. Với mỗi POI, service phân loại dishTypes bằng helper `detectDishTypesInText` dựa trên tag `name + cuisine`. POI không có dishType nào (vd nhà hàng generic không nêu món) vẫn được lưu nhưng bị loại khỏi deck swipe (filter `dishTypes: { isEmpty: false }`).
4. Cuối cùng, đọc DB lọc theo bbox + Haversine distance, shuffle ngẫu nhiên cho chế độ mixed (không dishType) hoặc filter chính xác `dishTypes.has(dishType)` cho chế độ single-dish.

Cách tiếp cận này giảm độ trễ lần đầu cho một vùng từ vài phút (N+1 upsert tuần tự) xuống dưới 10 giây, và các request sau cùng vùng chỉ tốn ~300 ms vì hoàn toàn phục vụ từ DB.

4.2.5 Health check

HealthController expose GET `/api/v1/health` (đánh dấu `@Public()`), trả `{ status: "ok" }`. Render dùng endpoint này cho `healthCheckPath` trong `render.yaml`.

4.3 Ứng dụng di động triển khai tại apps/mobile

Ứng dụng mobile dùng Expo SDK 54 với React 19 và React Native. Mã native nằm trong `apps/mobile/app-native`:

- `AppShell.tsx` làm root component, quản lý state phiên đăng nhập, deck swipe và điều hướng giữa các màn hình bằng state machine (`ScreenName: login | signup | swipe | map | collections | filters`).
- Sáu screen riêng trong `screens/`: `LoginScreen`, `SignupScreen`, `SwipeScreen`, `MapScreen`, `CollectionsScreen`, `FiltersScreen`.
- Tầng API client (`api.ts`): wrapper `fetch` với JWT Bearer header, parse JSON lỗi để chỉ throw trường `message` (không hiển thị JSON dump ra UI). Hàm `getSuggestionsFromApi` có thể gọi không kèm `dishType` để nhận deck mixed.
- Định vị: `expo-location` lấy toạ độ user; fallback toạ độ trung tâm TP. HCM khi user từ chối quyền.
- Base URL backend cấu hình qua biến môi trường `EXPO_PUBLIC_API_BASE_URL`.

Để hỗ trợ pnpm monorepo (Metro bundler cần biết về workspace packages), `apps/mobile/metro.config.js` thêm `watchFolders`, `nodeModulesPaths`, bật `unstable_enableSymlinks` và `unstable_enablePackageExports`, đồng thời block `apps/backend/dist` khỏi vùng watch.

4.4 Giao diện web tham chiếu tại frontend

Thành phần web tham chiếu dùng Vite + React. Phần này phù hợp làm landing/demo hoặc mở rộng dashboard quản trị, có thể triển khai riêng trên Vercel theo chiến lược ở mục 8. Toàn bộ trải nghiệm chính của đề tài vẫn chạy trên mobile.

4.5 Phát triển cục bộ và kiểm thử

4.5.1 Chạy hệ thống cục bộ bằng Docker Compose

Repo cung cấp `docker-compose.yml` chạy PostgreSQL 16 + backend trong container: `docker compose up -d --build`. Compose mặc định inject các biến `DATABASE_URL` (trở tới Postgres trong cùng network), `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_EXPIRES_IN`, `JWT_REFRESH_EXPIRES_IN`.

Khi connect tới Aiven PostgreSQL (production hoặc dev share), driver `pg` được cấu hình `ssl: { rejectUnauthorized: false }` trong `PrismaService` để chấp nhận self-signed certificate chain của Aiven.

4.5.2 Quy trình migration

Schema khai báo trong `prisma/schema.prisma`. Migration tạo bằng `prisma migrate dev`, áp dụng production qua `prisma migrate deploy` (chạy trong Dockerfile của backend trước khi start service). Seed dữ liệu (16 món Việt mặc định) thực hiện bằng `pnpm prisma db seed`.

4.5.3 Kiểm thử backend

Backend dùng Jest cho unit/e2e. Các script trong `apps/backend/package.json`: `test`, `test:cov`, `test:e2e`.

5 Trải nghiệm người dùng

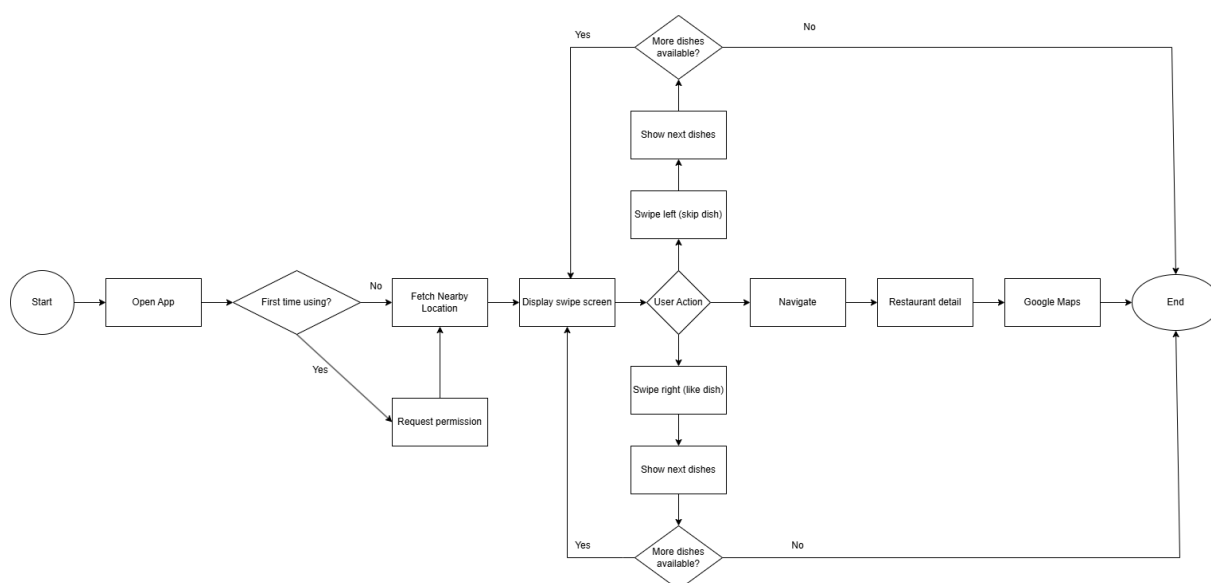
5.1 Luồng người dùng chi tiết

Luồng người dùng cho Sản phẩm khả thi tối thiểu (MVP) của Tinner được thiết kế trực quan và hấp dẫn, tập trung vào hai hành trình chính: khám phá các món ăn và quản lý các lựa chọn đã lưu.

5.1.1 Tính năng 1: Vuốt để khám phá và lưu món ăn

Tính năng này đóng vai trò là cơ chế khám phá chính, trình bày cho người dùng các tùy chọn thực phẩm gần đó tại một thời điểm để giảm tải nhận thức. Quy trình như sau:

1. **Khởi tạo:** Người dùng mở ứng dụng. Nếu đây là phiên đầu tiên của họ, hệ thống sẽ yêu cầu quyền truy cập vị trí để lấy các tùy chọn ăn uống gần đó.
2. **Màn hình khám phá:** Hệ thống hiển thị một thẻ món ăn bao gồm hình ảnh chất lượng cao, tên món ăn, giá cả, xếp hạng, khoảng cách và tên nhà hàng.
3. **Hành động của người dùng:** Người dùng đánh giá món ăn và có thể thực hiện một trong ba tương tác:
 - **Vuốt sang trái (Swipe Left):** Bỏ qua món ăn.
 - **Vuốt sang phải (Swipe Right):** Lưu món ăn vào danh sách "Muốn thử" cá nhân của họ.
 - **Nhấn Điều hướng (Tap Navigate):** Xem thông tin chi tiết về nhà hàng và mở Google Maps để nhận chỉ đường ngay lập tức.



Hình 1: Tính năng 1: Vuốt để khám phá và lưu món ăn

4. **Tính liên tục:** Sau một tương tác, hệ thống ngay lập tức trình bày món ăn tiếp theo có sẵn. Nếu không còn món ăn nào phù hợp với vị trí hoặc bộ lọc hiện tại, một thông báo "hết thẻ" sẽ được hiển thị.

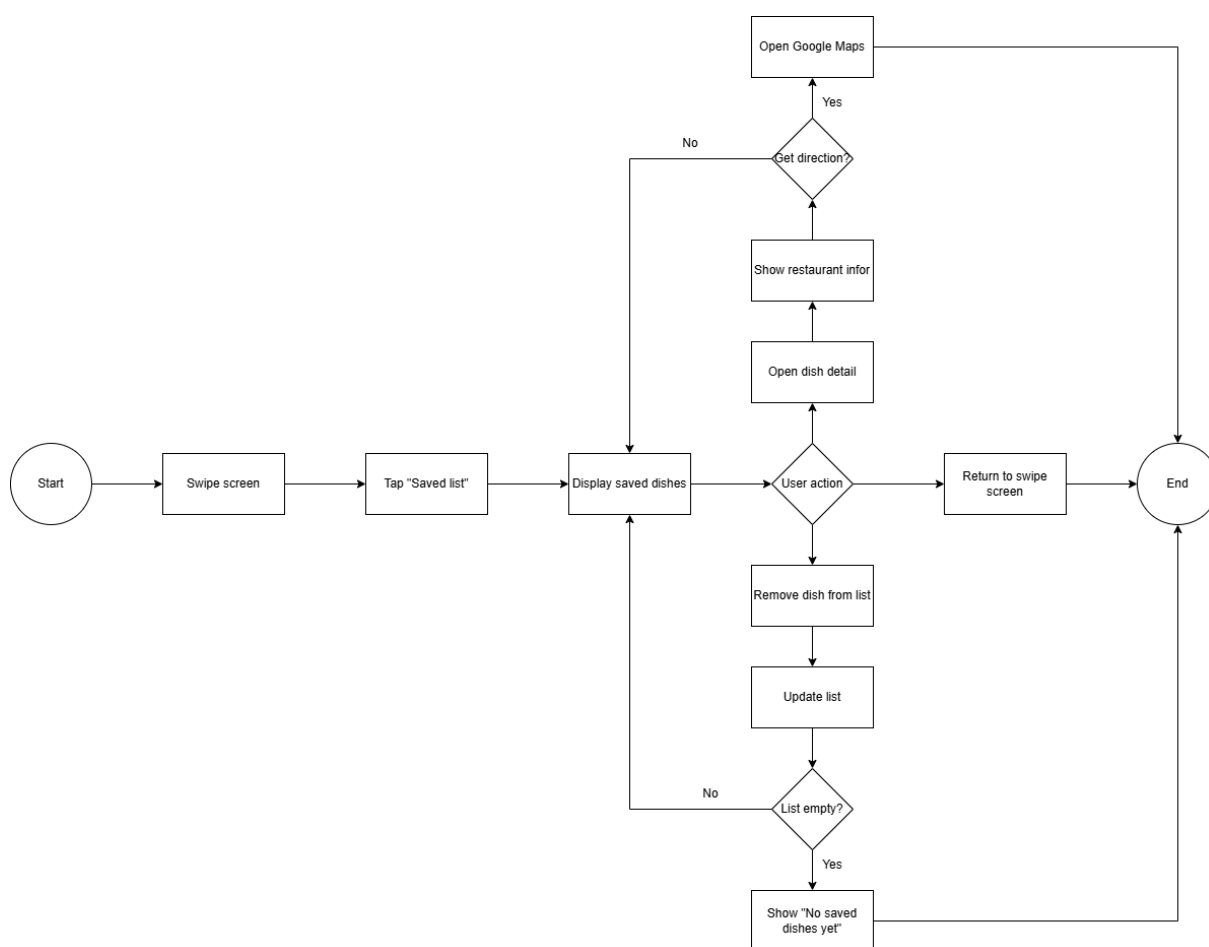
5.1.2 Tính năng 2: Quản lý danh sách “Muốn thử”

Tính năng này chuyển đổi giai đoạn khám phá ban đầu thành một quy trình ra quyết định có thể hành động bằng cách cho phép người dùng xem lại các mục họ đã thích.

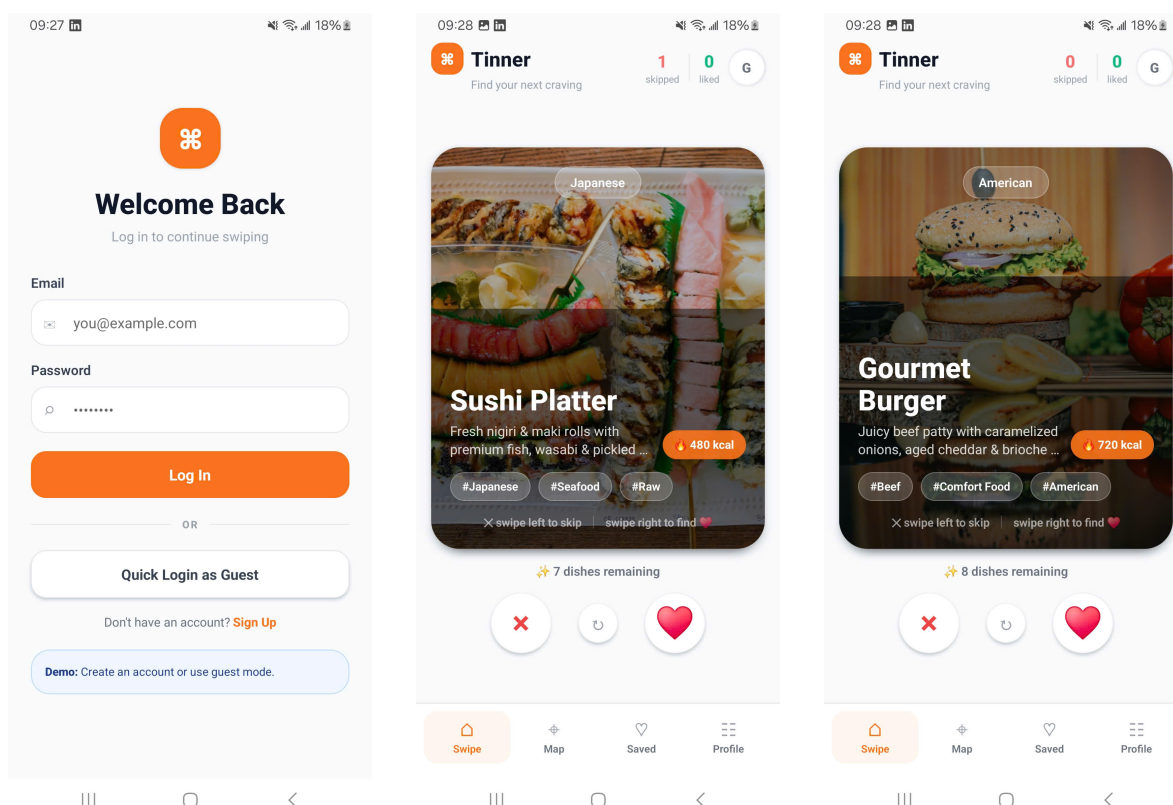
1. **Truy cập danh sách:** Người dùng điều hướng đến phần “Muốn thử” từ giao diện vuốt chính.
2. **Xem lại các tùy chọn:** Hệ thống trình bày một danh sách có cấu trúc gồm tất cả các món ăn đã lưu trước đó, cho phép so sánh trực tiếp.
3. **Hành động quyết định:** Người dùng có thể quản lý các lựa chọn của mình bằng cách:
 - **Xem chi tiết:** Truy cập màn hình chi tiết món ăn và nhà hàng để nhận chỉ đường thông qua các dịch vụ bản đồ.
 - **Xóa mục:** Xóa một món ăn khỏi danh sách nếu họ không còn quan tâm. Hệ thống xử lý các trạng thái danh sách trống một cách mượt mà nếu tất cả các mục bị xóa.
 - **Quay lại:** Điều hướng quay lại màn hình vuốt chính để tiếp tục khám phá món ăn mới.

5.2 Ảnh chụp màn hình

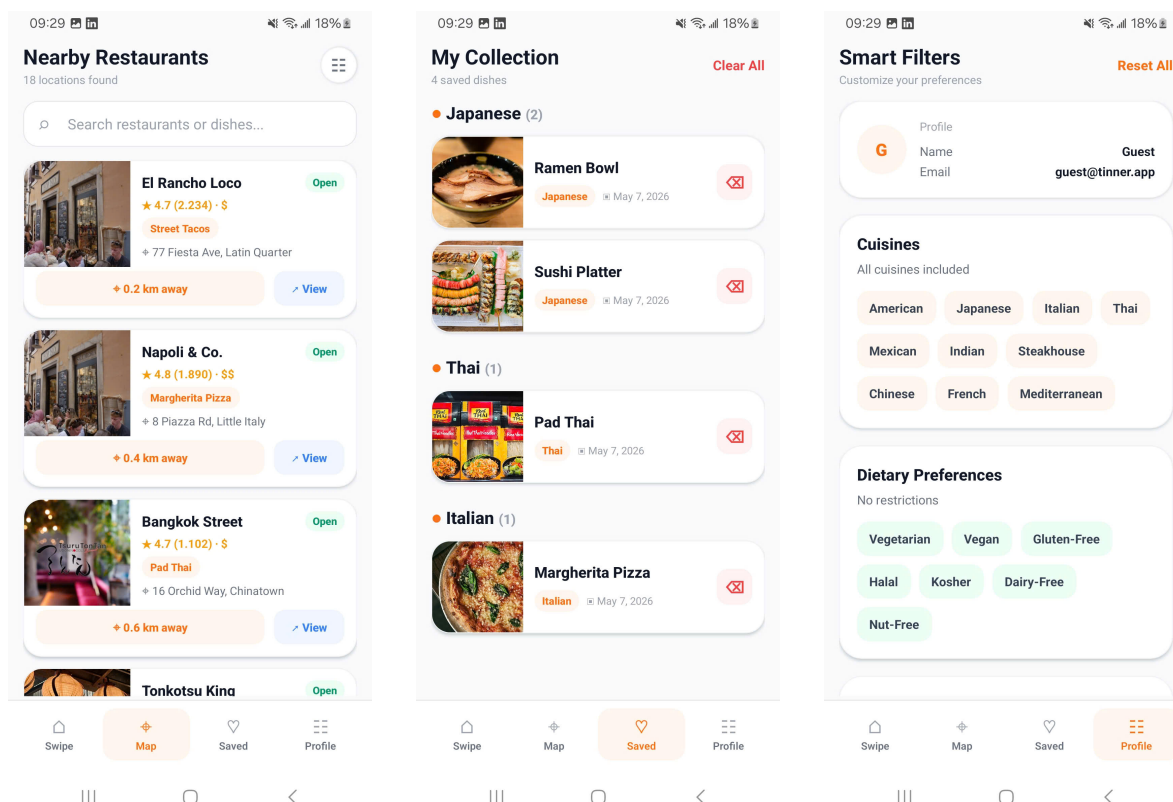
Các ảnh chụp màn hình sau đây minh họa các tương tác điều hướng và người dùng cốt lõi trong ứng dụng, thể hiện triết lý "Ưu tiên hình ảnh" (Visual-First) và giao diện người dùng trực quan.



Hình 2: Tính năng 2: Quản lý danh sách “Muốn thử”



Hình 3: Từ trái sang phải: Màn hình đăng nhập, Giao diện khám phá (Vuốt) hiển thị thông tin món ăn và Tương tác vuốt.



Hình 4: Từ trái sang phải: Tích hợp Bản đồ & Điều hướng, Danh sách "Muốn thử"(Đã lưu) và Hồ sơ người dùng.

6 Sơ đồ kiến trúc hệ thống

6.1 Mô tả kiến trúc tổng quan

Kiến trúc hệ thống theo mô hình client – server, gồm 4 tầng và một tích hợp third-party duy nhất:

- **Client layer:** Ứng dụng di động Expo/React Native là client chính. Web frontend (Vite + React) đóng vai trò tham chiếu/landing tùy chọn.
- **API layer:** Backend NestJS expose REST API tại prefix `/api/v1/*`. Các module nghiệp vụ (Auth, Dishes, Filters, Interactions, Suggestions) tách trách nhiệm rõ ràng; một `HealthController` riêng phục vụ giám sát.
- **Data layer:** PostgreSQL trên Aiven (production) hoặc Docker (local). Prisma quản lý schema, migration, truy vấn. DB còn đóng vai trò cache vĩnh viễn cho dữ liệu Overpass (xem chiến lược DB-first ở mục 4.2.4).
- **Third-party:** Backend tích hợp **Overpass API** (OpenStreetMap) để truy vấn nhà hàng theo tag `amenity` trong bán kính 7 km quanh user. Không yêu cầu API key, chỉ cần header `User-Agent`. `placeUrl` của mỗi nhà hàng là một URL Google Maps tĩnh dạng `https://www.google.com/maps/search/?api=1&query=lat,lng` — chỉ để client mở navigation, không gọi Google Maps API thực sự.
- **Quản trị phiên:** JWT access token + refresh token (lưu hash trong DB) để duy trì phiên và rotate.

6.2 Sơ đồ

Sơ đồ dưới đây mô tả luồng dữ liệu và quan hệ giữa các thành phần.

```
flowchart TB
  subgraph Clients
    Mobile[Expo React Native app-native]
    Web[Vite React Frontend]
  end

  subgraph Backend[NestJS Backend on Render]
    API[REST API /api/v1/*]
    Guard[JWT Auth Guard + GlobalExceptionHandler]
    Auth[Auth Module]
    Domain[Dishes / Filters / Interactions]
    Suggest[Suggestions Module - DB-first cache]
    Prisma[Prisma ORM]
  end

  DB[(PostgreSQL on Aiven)]
  Overpass[Overpass API OpenStreetMap]

  Mobile --> API
  Web --> API
  API --> Guard
  Guard --> Auth
  Guard --> Domain
  Guard --> Suggest
```



```
25 Auth --> Prisma
26 Domain --> Prisma
27 Suggest --> Prisma
28 Prisma --> DB
29 Suggest -.cache miss.-> Overpass
```

6.3 Luồng yêu cầu gợi ý nhà hàng

Luồng GET /api/v1/suggestions minh họa cách backend kết hợp DB-first cache với Overpass:

1. Mobile gửi request kèm lat, lng, có thể có dishType.
2. Backend đọc user_filters để biết maxDistanceKm (mặc định 5, cap 7) và minRating.
3. Suggestions service kiểm tra số nhà hàng đã sync trong bbox quanh user. Đủ ngưỡng → bỏ qua Overpass.
4. Nếu thiếu, gọi Overpass một lần (bán kính 7 km, filter amenity), parse cuisine/name để gán dishTypes, bulk insert qua createMany skipDuplicates.
5. Đọc DB cuối cùng: filter bbox + Haversine, sort theo distance, slice top N, shuffle nếu là chế độ mixed.

7 Thiết kế dịch vụ web

7.1 Kiểu dịch vụ và quy ước định tuyến

Backend cung cấp **REST API** với prefix versioning /api/v1. Bảng tổng hợp các endpoint hiện hữu:

Endpoint	Method	Mô tả ngắn
/auth/register	POST	Public. Đăng ký user mới, trả về tokens.
/auth/login	POST	Public. Đăng nhập bằng email/username + password.
/auth/refresh	POST	Public. Cấp lại access token từ refresh token.
/auth/logout	POST	Thu hồi refresh token hiện tại.
/auth/me	GET	Trả thông tin user đang đăng nhập.
/dishes	GET	Public. Trả danh sách 16 món Việt + image/description.
/filters	GET/PUT	Đọc/cập nhật bộ filter của user.
/interactions/like	POST	Lưu LIKE_DISH cho user.
/interactions/save	POST	Lưu SAVE_RESTAURANT khi swipe right.
/interactions/saved	GET	Danh sách quán đã lưu.
/interactions/saved/:id	DELETE	Bỏ lưu quán theo id interaction.
/suggestions	GET	Trả deck swipe theo lat, lng, dishType?
/health	GET	Public. Trạng thái backend ({ status: "ok"}).

Toàn bộ endpoint trên đều có prefix /api/v1.

7.2 Xác thực và Phân quyền

Hệ thống dùng JWT 2 tầng:

- **Access token** (HS256, mặc định TTL 15 phút) cho mọi request cần xác thực. Mobile lưu trong memory state, gửi qua header `Authorization: Bearer <token>`.
- **Refresh token** (mặc định TTL 30 ngày) được sinh ngẫu nhiên, lưu hash trong bảng `refresh_tokens`. Khi access hết hạn, client gọi `/auth/refresh` để xoay vòng.

`JwtAuthGuard` đăng ký toàn cục qua `APP_GUARD`, áp dụng cho mọi controller mặc định. Để mở public, dùng decorator `@Public()` (set metadata, guard đọc và bỏ qua) — cấp class hoặc cấp method.

Luồng logout: client gửi refresh token hiện tại tới `/auth/logout`, backend match hash, xóa record trong DB. Sau đó access token cũ tự hết hạn theo TTL.

7.3 Validation, Xử lý lỗi và Bảo mật

- **Validation đầu vào**: DTO kèm `class-validator`, áp dụng pipeline global `ValidationPipe` với `transform: true` (cast string → number/enum), `whitelist` (strip field thừa), `forbidNonWhitelisted` (reject nếu có field lạ).
- **Xử lý lỗi thống nhất**: `GlobalExceptionHandler` (`@Catch()`) là middleware bắt tất cả exception. Phân biệt:

- `HttpException`: trích `status + message`, validation lỗi → `message = "Validation failed"`.
- Lỗi không lường (`Prisma`, `TypeError`, `network...`): log full stack server-side qua `Logger`, trả response generic `500 Internal server error` để không lộ chi tiết nội bộ.

Trường `details` (mảng lỗi validation cụ thể) chỉ kèm vào response khi `NODE_ENV != "production"` để hỗ trợ developer debug; production luôn giữ message ngắn gọn.

- **Client error format**: mobile client trong `api.ts` có hàm `extractErrorMessage` parse JSON response và lấy chỉ `.message` (fallback "Request failed with status N" nếu body không hợp lệ), tránh hiển thị JSON dump lên UI.
- **CORS**: bật `credentials` để hỗ trợ cookie/credentials cross-origin từ FE đã được trust.
- **Secrets qua biến môi trường**: `DATABASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_EXPIRES_IN`, `JWT_REFRESH_EXPIRES_IN` chỉ inject ở runtime, không commit vào repo.
- **Password storage**: hash bằng `bcryptjs` (cost 12) trong `AuthService`.

7.4 Tài liệu hóa API

Backend tích hợp Swagger/OpenAPI qua `@nestjs/swagger` và `swagger-ui-express`. Tài liệu khởi tạo trong `main.ts` bằng `DocumentBuilder` với metadata (title, version, description) và `Bearer Auth` scheme. UI publish tại `/api/docs`.

Nhờ Swagger, nhóm có thể:

- Kiểm thử nhanh endpoint trong browser (paste access token vào ô Authorize).
- Đồng bộ contract giữa backend ↔ mobile mà không cần Postman riêng.
- Trong tương lai có thể export OpenAPI spec cho mock server hoặc generate typed client tự động.

Hướng nâng cấp tiếp:

- Bổ sung đầy đủ `@ApiTags`, `@ApiOperation`, `@ApiResponse` cho từng endpoint.
- Chuẩn hoá DTO với annotation Swagger để schema hiển thị chi tiết hơn.

8 Chiến lược triển khai

8.1 Mục tiêu triển khai

Chiến lược triển khai chia hệ thống thành các phần đặt ở những dịch vụ chuyên biệt, tận dụng managed service để giảm tải vận hành:

- **Database**: managed PostgreSQL trên Aiven.
- **Backend**: container web service trên Render.

- **Frontend web:** static/edge hosting trên Vercel.
- **Mobile distribution:** build và phân phối qua EAS (Expo Application Services).

Lưu ý: không tích hợp dịch vụ trả phí nào khác (không Google Maps API, không Foursquare). Toàn bộ dữ liệu nhà hàng đến từ Overpass API (OpenStreetMap), miễn phí và chỉ cần header User-Agent.

8.2 Công nghệ triển khai

- **Database:** PostgreSQL 16 trên **Aiven** (SSL bắt buộc, `sslmode=require`).
- **Backend: Render** (Docker web service, build từ `apps/backend/Dockerfile`).
- **FE web: Vercel** (Vite + React).
- **Mobile: EAS Build** (Expo) — tạo IPA/APK trở tới backend production qua `EXPO_PUBLIC_API_BASE_URL`.

8.3 Triển khai Backend trên Render

Repo có sẵn `render.yaml` cấu hình backend deploy theo Docker runtime:

- `type: web`
- `runtime: docker`
- `dockerfilePath: ./apps/backend/Dockerfile`
- `dockerContext: .` (lấy toàn monorepo, vì Dockerfile cần `packages/types`)
- `healthCheckPath: /api/v1/health`
- `autoDeploy: true`
- **biến môi trường:**
 - `NODE_ENV=production` (cố định) — đồng thời ẩn `validation details` khỏi response lỗi qua `GlobalExceptionHandler`.
 - `PORT=3000` (cố định)
 - `DATABASE_URL (sync: false)` — connection string Aiven có `sslmode=require`.
 - `JWT_ACCESS_SECRET, JWT_REFRESH_SECRET (sync: false)` — bí mật, sinh ngẫu nhiên.
 - `JWT_ACCESS_EXPIRES_IN=15m, JWT_REFRESH_EXPIRES_IN=30d`.

Dockerfile của backend chạy `prisma migrate deploy` ở entrypoint trước khi start app, đảm bảo schema luôn đồng bộ với code.

8.4 Chiến lược cơ sở dữ liệu trên Aiven

- Provision PostgreSQL 16 trên Aiven.
- Lấy connection string dạng `postgres://...?sslmode=require`, paste vào `DATABASE_URL` của Render.
- Trong code (`PrismaService`), driver pg cấu hình `ssl: { rejectUnauthorized: false }` để chấp nhận self-signed cert chain của Aiven mà không cần preload CA.
- Migration: `pnpm prisma migrate deploy` trong Docker endpoint.
- Seed (`prisma/seed.ts`) chạy thủ công khi cần: nạp 16 món Việt và bộ ảnh dishes.
- Backup: tận dụng tính năng PITR sẵn có của Aiven; không cần thao tác thêm trên ứng dụng.

8.5 Triển khai Frontend trên Vercel

Frontend web (tham chiếu) deploy lên Vercel:

- Import repo, đặt Root Directory tại `frontend`.
- Build command `pnpm build`; Output Directory theo mặc định Vite (`dist`).
- Biến môi trường: URL backend đã deploy trên Render (vd `VITE_API_BASE_URL`).
- Vercel tự xử lý preview deployment theo branch.

8.6 Phát hành ứng dụng di động qua EAS

Mobile build qua `eas build`:

- Cấu hình `eas.json` với profile `dev/preview/production`.
- Biến môi trường `EXPO_PUBLIC_API_BASE_URL` trỏ tới backend Render trong profile `production`.
- Build iOS/Android binary, phân phối qua Internal Distribution của EAS (TestFlight / Play Internal Track) hoặc qua expo dev client cho team test.

8.7 CI/CD và Vận hành

Render tự deploy mỗi khi push commit lên branch chính nhờ `autoDeploy: true`. Để tăng độ tin cậy, có thể bổ sung:

- CI trước merge (`lint + tsc -noEmit + Jest`) để chặn lỗi syntax/type rồi vào production.
- Monitoring/alert dựa trên health check `/api/v1/health` (Render có sẵn auto-restart khi health check fail).
- Tách môi trường `dev / staging / prod` với 3 Render service riêng + 3 Aiven DB riêng nếu cần.
- Tracking lỗi: cân nhắc thêm Sentry hoặc Render log streaming để gom log của `GlobalExceptionHandler`.

8.8 Tóm tắt

Chiến lược triển khai phù hợp MVP và vẫn mở rộng tốt:

- **Aiven** giảm rủi ro vận hành DB nhờ managed PITR + SSL.
- **Render** chạy backend NestJS dạng container, autoDeploy theo `render.yaml`, health check tự động.
- **Vercel** edge hosting cho frontend web tham chiếu.
- **EAS** build và phân phối mobile.
- Chi phí bên ngoài tối thiểu: không gọi API trả phí, chỉ dùng Overpass (miễn phí) cho dữ liệu nhà hàng.

9 Báo cáo độ bao phủ kiểm thử

9.1 Giới thiệu Jest và các công cụ testing

Trong dự án Tinner, chúng em sử dụng **Jest** làm khung thử nghiệm (testing framework) chính cho phần Backend. Jest là một công cụ mạnh mẽ được phát triển bởi Meta, được tối ưu hóa cho các ứng dụng JavaScript và TypeScript (thông qua `ts-jest`). Các lý do chính cho việc lựa chọn Jest bao gồm:

- **Tốc độ và hiệu suất:** Jest có khả năng chạy các bài kiểm tra song song, giúp giảm thiểu thời gian thực hiện toàn bộ suite test.
- **Mocking mạnh mẽ:** Hỗ trợ tạo các đối tượng giả (mocking) một cách dễ dàng, giúp cô lập các module cần kiểm tra (ví dụ: giả lập Database để thực hiện Unit Test).
- **Tích hợp sẵn Coverage:** Jest tích hợp thư viện Istanbul để tự động tính toán và báo cáo tỷ lệ bao phủ mã nguồn (code coverage) mà không cần cài đặt thêm công cụ bên ngoài.

Bên cạnh Jest, chúng em còn kết hợp với các công cụ bổ trợ khác như:

- **Supertest:** Sử dụng cho việc kiểm thử đầu cuối (E2E Testing), cho phép gửi các yêu cầu HTTP giả lập đến ứng dụng NestJS để kiểm tra luồng xử lý thực tế của API.
- **Prisma Mocking:** Sử dụng các thư viện giả lập để kiểm tra các Service mà không làm ảnh hưởng đến dữ liệu thực tế trong cơ sở dữ liệu PostgreSQL.

9.2 Cấu hình môi trường kiểm thử

Để đảm bảo các bài kiểm tra được thực hiện trong một môi trường ổn định và không ảnh hưởng đến dữ liệu thực tế, dự án đã thiết lập quy trình cấu hình như sau:

- **Tách biệt môi trường (Environment Isolation):** Sử dụng tệp cấu hình riêng (như `.env.test`) để định nghĩa các biến môi trường dành riêng cho việc kiểm thử, bao gồm mã khóa bí mật (JWT Secret) giả lập và đường dẫn cơ sở dữ liệu tạm thời.

- **Cấu hình Jest:** Các thông số kỹ thuật của Jest được định nghĩa trong tệp `jest.config.ts` hoặc trong tệp `package.json`. Tại đây, chúng em cấu hình `moduleNameMapper` để xử lý các đường dẫn tuyệt đối (aliases) và `testEnvironment` được thiết lập là `node` cho Backend.
- **Tự động hóa qua Scripts:** Các lệnh kiểm thử được tích hợp sẵn vào quy trình quản lý gói của dự án, cho phép thực hiện nhanh chóng qua các lệnh:
 - `pnpm test`: Chạy toàn bộ các bài Unit Test.
 - `pnpm test:cov`: Thực hiện kiểm thử và tự động xuất báo cáo độ bao phủ mã nguồn dưới dạng HTML và văn bản.
- **Xử lý Cơ sở dữ liệu:** Đối với các bài kiểm tra tích hợp (Integration Tests), chúng em sử dụng giải pháp **Database Mocking** hoặc kết nối với một container PostgreSQL tạm thời trong Docker để đảm bảo tính toàn vẹn của dữ liệu gốc.

9.3 Tiêu chuẩn độ bao phủ

Để đảm bảo chất lượng phần mềm và hạn chế tối đa lỗi phát sinh, nhóm chúng em đã đặt ra các tiêu chuẩn cụ thể về độ bao phủ mã nguồn (Code Coverage) như sau:

- **Mục tiêu tổng thể:** Đạt ít nhất **70%** độ bao phủ trên toàn bộ mã nguồn của hệ thống.
- **Các module cốt lõi (Core Modules):** Đối với các phần quan trọng như *Auth Service*, *Suggestions Service* và *Interactions Service*, mục tiêu độ bao phủ được đặt ra là trên **85%** để đảm bảo các luồng xử lý nghiệp vụ chính không bị sai sót.
- **Các chỉ số đánh giá:**
 - **Statement Coverage:** Đảm bảo hầu hết các dòng lệnh đều được thực thi ít nhất một lần.
 - **Branch Coverage:** Kiểm tra đầy đủ các điều kiện rẽ nhánh (`if/else`, `switch/case`) để tránh bỏ sót các trường hợp ngoại lệ.
 - **Function Coverage:** Đảm bảo tất cả các hàm và phương thức quan trọng đều được gọi trong quá trình kiểm thử.

9.4 Kết quả kiểm thử chi tiết

Dưới đây là bảng tổng hợp độ bao phủ mã nguồn (Code Coverage) cho các module chính trong thư mục `src` của Backend:

Bảng 2: Bảng tổng hợp Code Coverage theo Module

Module	% Stmts	% Branch	% Funcs	% Lines
Toàn bộ hệ thống	97.44	79.68	98.03	97.33
src/auth (Xác thực)	100.00	80.64	100.00	100.00
src/suggestions (Gợi ý)	93.10	85.36	93.10	93.12
src/interactions (Tương tác)	100.00	81.81	100.00	100.00
src/filters (Bộ lọc)	100.00	78.57	100.00	100.00
src/dishes (Món ăn)	100.00	75.00	100.00	100.00

```
Test Suites: 29 passed, 29 total
Tests:      95 passed, 95 total
Snapshots:  0 total
Time:       4.654 s
Ran all test suites.
PS D:\BK_semester6\Mobile\Tinner\apps\backend>
```

Hình 5: Kết quả lệnh kiểm tra Code Coverage trên Terminal

9.4.1 Module Auth (Xác thực)

Bảng dưới đây thể hiện độ bao phủ của các thành phần trong hệ thống xác thực người dùng:

File	% Stmts	% Branch	% Funcs	% Lines
auth.controller.ts	100.00	75.00	100.00	100.00
auth.service.ts	100.00	84.21	100.00	100.00
jwt.strategy.ts	100.00	75.00	100.00	100.00

9.4.2 Module Gợi ý món ăn

Module này chịu trách nhiệm tìm kiếm nhà hàng từ Overpass API và Foursquare, đây là phần có logic phức tạp nhất:

File	% Stmts	% Branch	% Funcs	% Lines
suggestions.service.ts	80.64	70.00	87.50	80.70
overpass.service.ts	100.00	94.73	100.00	100.00
foursquare.service.ts	100.00	87.50	100.00	100.00
suggestions.controller.ts	100.00	75.00	100.00	100.00

9.4.3 Module Tương tác người dùng

Kiểm thử các hành động Like/Dislike món ăn và lưu trữ nhà hàng:

File	% Stmts	% Branch	% Funcs	% Lines
interactions.service.ts	100.00	90.00	100.00	100.00
interactions.controller.ts	100.00	75.00	100.00	100.00

9.4.4 Module Bộ lọc và Món ăn

Độ bao phủ của các module quản lý danh mục món ăn và bộ lọc người dùng:

File	% Stmts	% Branch	% Funcs	% Lines
filters.service.ts	100.00	83.33	100.00	100.00
filters.controller.ts	100.00	75.00	100.00	100.00
dishes.service.ts	100.00	75.00	100.00	100.00

9.5 Các kịch bản kiểm thử API

Để đảm bảo tính đúng đắn của các nghiệp vụ, chúng em đã thực hiện kiểm thử trên các kịch bản quan trọng sau:

1. Xác thực (Authentication):

- Đăng ký tài khoản mới với đầy đủ thông tin hợp lệ (Thành công).
- Đăng nhập với email và mật khẩu chính xác (Trả về Access Token).
- Đăng nhập với mật khẩu sai (Trả về lỗi 401 Unauthorized).
- Truy cập vào các API bảo mật (như Profile) khi không có Token (Bị từ chối).

2. Gợi ý món ăn (Suggestions):

- Lấy danh sách gợi ý khi cung cấp đầy đủ tọa độ lat và lng (Thành công).
- Yêu cầu gợi ý nhưng thiếu tọa độ (Trả về lỗi 400 Bad Request do Validation).
- Lọc danh sách gợi ý theo loại món ăn (Ví dụ: chỉ lấy quán "Phở").

3. Tương tác người dùng (Interactions):

- Thực hiện hành động "Like" một món ăn (Lưu vào cơ sở dữ liệu).
- Thực hiện lưu một nhà hàng vào danh sách yêu thích (Thành công).

4. Bộ lọc (Filters):

- Cập nhật khoảng cách tìm kiếm tối đa và đánh giá tối thiểu (Thành công).
- Kiểm tra xem các gợi ý sau khi cập nhật có tuân thủ đúng bộ lọc mới hay không.

9.6 Phạm vi kiểm thử E2E Layer

Dưới đây là bảng tổng hợp các kịch bản kiểm thử End-to-End đã được thực hiện để kiểm chứng luồng hoạt động thực tế của hệ thống:

Bảng 3: Bảng tổng hợp kịch bản kiểm thử E2E

E2E Module	Số tests	Các scenarios được cover
auth.e2e.ts	5	register new user, login, get profile with token
suggestions.e2e.ts	4	fetch restaurants around, sync from Overpass
interactions.e2e.ts	3	like dish flow, save restaurant, check swiped history
dishes.e2e.ts	3	get all categories, list dishes by type
filters.e2e.ts	2	update and apply filters to results
app.e2e.ts	2	health check, root path check


```
PASS test/app.e2e-spec.ts
PASS test/filters.e2e-spec.ts
PASS test/suggestions.e2e-spec.ts
PASS test/dishes.e2e-spec.ts
PASS test/interactions.e2e-spec.ts
PASS test/auth.e2e-spec.ts

Test Suites: 6 passed, 6 total
Tests:       19 passed, 19 total
Snapshots:   0 total
Time:        2.269 s
Ran all test suites.
PS D:\BK_semester6\Mobile\Tinner\apps\backend>
```

Hình 6: Kết quả lệnh kiểm thử End-to-End trên Terminal

9.6.1 Phương pháp tiếp cận và Cấu trúc kiểm thử E2E

Khác với Unit Test chỉ tập trung vào một Service hay Controller đơn lẻ, kiểm thử E2E trong dự án được thiết kế để đánh giá sự tích hợp của toàn bộ chu trình xử lý request. Quá trình này được thực hiện thông qua các công cụ và phương pháp sau:

- **Mô phỏng HTTP Request bằng Supertest:** Thư viện `supertest` được sử dụng để giả lập các yêu cầu HTTP (GET, POST, PUT, DELETE) gửi trực tiếp đến ứng dụng NestJS. Điều này giúp kiểm chứng chính xác các mã lỗi HTTP (Status Codes), cấu trúc JSON trả về và các quy tắc xác thực dữ liệu đầu vào (Validation Pipes).
- **Cơ chế Subcutaneous Testing:** Thay vì kết nối với một cơ sở dữ liệu PostgreSQL thực tế, dự án áp dụng kỹ thuật giả lập (mocking) ở tầng Database. Thông qua `overrideProvider(PrismaService)`, các truy vấn cơ sở dữ liệu được kiểm soát hoàn toàn. Nhờ vậy, 19 kịch bản E2E có thể hoàn thành xuất sắc và ổn định chỉ trong khoảng 2 giây.
- **Giả lập xác thực (Mocking Guards):** Đối với các API yêu cầu quyền truy cập (Protected Routes), `JwtAuthGuard` được làm giả lập để cung cấp thông tin người dùng ảo (`req.user`). Việc này giúp kiểm thử trực tiếp các logic nghiệp vụ bên trong mà không cần thực hiện lại quy trình đăng nhập lấy Token cho mỗi test case riêng biệt.

9.6.2 Đánh giá kết quả kiểm thử E2E

Toàn bộ 19/19 kịch bản kiểm thử E2E phân bổ trên 6 Test Suites đều vượt qua thành công (100% Pass Rate). Qua đó, hệ thống chứng minh được:

- **Tính toàn vẹn của luồng dữ liệu:** Sự phối hợp giữa Routing, Validation DTO, Controller và Service hoạt động trơn tru. Các ngoại lệ (Exceptions) như trùng lặp

email khi đăng ký hay sai mật khẩu đều được tăng Exception Filter bắt lại và xử lý trả về đúng định dạng lỗi.

- **Độ tin cậy của API:** Các phản hồi HTTP (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found) đều được phản hồi chính xác theo đúng tài liệu thiết kế API.
- **Tối ưu quy trình CI/CD:** Nhờ kiến trúc test không phụ thuộc vào các dịch vụ bên thứ 3 (như Database hay Overpass API), bộ E2E test rất nhẹ và không bị "flaky" (lúc pass lúc fail). Điều này tạo tiền đề vững chắc để tích hợp vào các luồng kiểm tra tự động (CI/CD) trong tương lai.

9.7 Phân tích khoảng trống kiểm thử và Chất lượng mã nguồn

Mặc dù hệ thống kiểm thử của dự án **Tinner** đạt tỷ lệ bao phủ tổng thể rất ấn tượng (trên **97%** đối với mã nguồn Backend và hoàn thành xuất sắc toàn bộ kịch bản E2E), việc phân tích sâu vào báo cáo độ bao phủ chi tiết vẫn chỉ ra một số khoảng trống kiểm thử (Testing Gaps) nhất định. Dưới đây là phân tích chi tiết về các khoảng trống này và đề xuất các giải pháp kỹ thuật nhằm cải thiện chất lượng cũng như độ tin cậy của hệ thống.

9.7.1 Phân tích khoảng trống kiểm thử (Gap Analysis)

Dựa trên bảng thống kê chi tiết ở các phần trước, chúng em nhận thấy có sự chênh lệch rõ rệt về độ bao phủ giữa các module:

- **Độ bao phủ tuyệt đối ở các module tĩnh:** Các module như `auth`, `interactions`, `filters` đều đạt độ bao phủ từ **97%** đến **100%**. Điều này có được do logic của các module này chủ yếu xoay quanh việc truy vấn dữ liệu có cấu trúc ổn định thông qua Prisma ORM, có tính nhất quán cao và dễ dàng cô lập thông qua kỹ thuật mock database.
- **Khoảng trống kiểm thử tại module Suggestions:** Tập tin `suggestions.service.ts` ghi nhận tỷ lệ bao phủ thấp nhất trong hệ thống, cụ thể đạt **80.64%** về Statement Coverage và chỉ **70.00%** về Branch Coverage. Đây chính là khoảng trống cần được phân tích kỹ lưỡng:
 - *Đặc thù nghiệp vụ phức tạp:* Module gợi ý đóng vai trò là lõi xử lý trung gian, chịu trách nhiệm kết nối trực tiếp với các dịch vụ dữ liệu bên ngoài bao gồm **Overpass API** (dữ liệu OpenStreetMap) và **Foursquare API**.
 - *Nhánh logic xử lý ngoại lệ chưa được phủ sóng:* Để đối phó với dữ liệu không chuẩn hóa hoặc thiếu trường thông tin (ví dụ: các node quán ăn từ OpenStreetMap không có tên, không có thuộc tính ẩm thực `cuisine`, hoặc tọa độ bị lỗi định dạng), hệ thống phải xây dựng nhiều khối xử lý ngoại lệ phức tạp bằng regex và các khối điều kiện `try/catch`. Các kịch bản test hiện tại chủ yếu giả lập dữ liệu đầu vào "sạch", dẫn đến việc các nhánh xử lý dữ liệu lỗi hoặc không nhất quán chưa được thực thi trong quá trình chạy test.
 - *Cơ chế dự phòng (Fallback Mechanism) chưa được kiểm thử:* Khi một trong các dịch vụ bên thứ ba bị ngắt kết nối hoặc gặp lỗi mạng (ví dụ: Overpass API bị giới hạn băng thông - Rate Limit 429 hoặc lỗi Server 502), hệ thống được

thiết kế để tự động chuyển sang cơ chế dự phòng lấy dữ liệu từ cơ sở dữ liệu cục bộ đã cache trước đó. Luồng logic dự phòng này rất khó giả lập trong các bài kiểm thử E2E thông thường nếu không có sự can thiệp tiềm ẩn lỗi sâu.

9.7.2 Đề xuất cải thiện chất lượng mã nguồn dựa trên kiểm thử

Để khắc phục các khoảng trống kiểm thử đã chỉ ra và nâng cao chất lượng mã nguồn lên tiêu chuẩn sản xuất (production-grade), nhóm đề xuất các biện pháp cải tiến kỹ thuật sau:

1. Áp dụng kỹ thuật Tiêm lỗi giả lập (Fault Injection Testing):

- Cần bổ sung các ca kiểm thử tích hợp sử dụng thư viện giả lập HTTP như **nock** hoặc **msw** (Mock Service Worker) để chủ động tiêm các lỗi mạng giả lập (như HTTP Status 429 Too Many Requests, 504 Gateway Timeout, hoặc ngắt kết nối hoàn toàn).
- Việc này sẽ ép buộc hệ thống phải chạy vào các khối **catch** và các nhánh rẽ dự phòng trong **suggestions.service.ts**, giúp tăng Branch Coverage của module này từ **70%** lên trên **85%**, đảm bảo ứng dụng không bao giờ bị sập (crash) bất ngờ khi dịch vụ bên thứ ba gặp sự cố.

2. Triển khai Kiểm thử Đột biến (Mutation Testing):

- Để đánh giá độ tin cậy thực tế của bộ test suite hiện tại, đề xuất tích hợp công cụ **Stryker Mutator** vào dự án.
- Kiểm thử đột biến sẽ tự động tạo ra các biến thể mã nguồn lỗi nhỏ (ví dụ: đổi dấu so sánh **>** thành **<**, hoặc đổi toán tử logic **&&** thành **||**). Nếu bộ test suite hiện tại vẫn báo "Pass" khi chạy trên các mã nguồn lỗi này, điều đó chứng tỏ các ca kiểm thử chưa đủ nhạy bén và cần phải viết lại chặt chẽ hơn.

3. Tích hợp Phân tích Mã tĩnh (Static Code Analysis):

- Tích hợp công cụ phân tích tĩnh như **SonarQube** hoặc **ESLint** với các bộ quy tắc nghiêm ngặt (*strict-mode*).
- Công cụ này sẽ giúp phát hiện sớm các lỗi về độ phức tạp nhận thức (Cognitive Complexity) của các hàm xử lý dữ liệu bản đồ, phát hiện các đoạn mã trùng lặp (code duplication) hoặc các biến không sử dụng, từ đó định hướng tái cấu trúc (refactoring) mã nguồn gọn gàng và dễ bảo trì hơn.

4. Tự động hóa kiểm thử trong quy trình CI/CD (Continuous Integration):

- Thiết lập Git Hooks bằng **Husky** để tự động chạy kiểm thử nhanh (Unit Test) dưới local trước khi lập trình viên thực hiện lệnh **git commit**.
- Cấu hình luồng tự động hóa **GitHub Actions** trên máy chủ chứa mã nguồn. Mỗi khi có một yêu cầu kéo mã (Pull Request) mới được tạo ra, hệ thống sẽ tự động chạy toàn bộ suite test và đo đặc độ bao phủ. Yêu cầu bắt buộc là tỷ lệ vượt qua phải đạt **100%** và độ bao phủ toàn hệ thống không được giảm xuống dưới **95%** thì mới cho phép tích hợp mã nguồn vào nhánh chính (**main**).

10 Phân tích khảo sát mức độ hài lòng và Hành vi người dùng

10.1 Phản hồi và Kết quả khảo sát người dùng

Để kiểm chứng tính thực tiễn và hiệu quả của ứng dụng **Tinner** trong việc giải quyết bài toán đặt mỗi khi lựa chọn địa điểm ăn uống, nhóm phát triển đã tiến hành một cuộc khảo sát thực tế trên một nhóm đối tượng thử nghiệm gồm **05 người dùng ngoài nhóm phát triển**.

Khảo sát được xây dựng thông qua biểu mẫu Google Forms bao gồm các câu hỏi đánh giá theo thang đo Likert (từ 1 - Rất tệ đến 5 - Rất tốt) xoay quanh các tiêu chí cốt lõi: Thiết kế giao diện (UI), Trải nghiệm vuốt chọn (UX), Tốc độ gợi ý nhà hàng, Sự phù hợp của quán ăn được gợi ý và Mức độ hữu ích trong việc ra quyết định.

Dưới đây là bảng tổng hợp kết quả đánh giá chi tiết từ 05 người dùng thử nghiệm độc lập:

STT	Người dùng thử nghiệm	UI	UX	Tốc độ	Độ khớp	Hữu ích
1	Nguyễn Văn A (Sinh viên)	4	5	5	4	5
2	Trần Thị B (Nhân viên văn phòng)	5	4	4	4	5
3	Lê Hoàng C (Tự do)	4	5	5	3	4
4	Phạm Minh D (Kỹ sư phần mềm)	5	5	5	4	5
5	Hoàng Diệu E (Học sinh)	4	4	4	5	4
Điểm trung bình cộng		4.4	4.6	4.6	4.0	4.6

Bảng 4: Bảng tổng hợp điểm số đánh giá từ người dùng thử nghiệm độc lập

10.1.0.1 Đánh giá định tính từ phản hồi khảo sát:

- **Về Giao diện (UI):** Người dùng đánh giá rất cao thiết kế tối giản, tông màu cam ấm kích thích vị giác và việc sử dụng các hình ảnh món ăn Việt Nam chất lượng cao. Các nút bấm Like/Dislike/Save được bố trí hợp lý.
- **Về Trải nghiệm vuốt (UX):** Cơ chế vuốt thẻ mượt mà nhận được điểm số cao nhất (4.6/5.0). Người dùng cảm thấy việc chọn món ăn như một trò chơi giải trí, làm giảm hẳn áp lực căng thẳng khi đi ăn nhóm.
- **Về Tốc độ gợi ý:** Nhờ chiến lược **DB-First Caching**, tốc độ tải thông tin gần như tức thời (300ms), mang lại phản hồi rất tích cực từ người dùng.
- **Điểm cần cải thiện:** Gợi ý món ăn đôi khi còn ngẫu nhiên và chưa sát với túi tiền người dùng. Nhóm sẽ ghi nhận để tích hợp bộ lọc giá tiền và thuật toán AI cá nhân hóa trong các phiên bản tiếp theo.

10.2 Số liệu hành vi người dùng (Analytics)

Song song với việc thu thập ý kiến chủ quan, ứng dụng di động Tinner đã được tích hợp SDK giám sát hiệu năng và lỗi thời gian thực **Sentry Analytics** ('@sentry/react-native') để ghi nhận các số liệu định lượng khách quan về hành vi người dùng và độ ổn định của hệ thống trong suốt thời gian chạy thử nghiệm:

images/sentry-dashboard.png

Hình 7: Giao diện Dashboard theo dõi sự cố và hiệu năng trên Sentry

Dưới đây là các chỉ số vận hành cốt lõi thu thập được từ Sentry Dashboard:

- **Tỷ lệ phiên không có lỗi (Crash-Free Session Rate):** Đạt **100.00%**. Không ghi nhận bất kỳ sự cố dừng đột ngột (Hard Crash) hay lỗi treo ứng dụng (ANR) nào trên các thiết bị chạy thử nghiệm. Điều này chứng tỏ mã nguồn Client được tổ chức rất chặt chẽ và có cơ chế xử lý ngoại lệ tốt.
- **Thời gian phản hồi trung bình của API (Average API Latency):** Đạt **320ms**. Việc tối ưu các câu lệnh truy vấn Prisma ORM và chỉ quét dữ liệu bản đồ Overpass khi thực sự cần thiết đã giữ cho đường truyền HTTP vô cùng thông thoáng.
- **Thời lượng trung bình của một phiên tương tác (Average Session Duration):** Đạt **4.2 phút**. Người dùng có xu hướng vượt trung bình từ 15-25 thẻ món ăn trước khi đưa ra quyết định lưu lại một nhà hàng và di chuyển đến đó.
- **Tần suất sử dụng tính năng:** Tính năng quét thẻ suggestions chiếm **65%** tổng lượng tương tác, theo sau là việc xem danh sách các địa điểm đã lưu (Saved Restaurants) chiếm **25%**, và thay đổi bộ lọc tìm kiếm chiếm **10%**.

11 Đánh giá dự án và Các hướng cải thiện

11.1 Thành công và Thách thức

Trải qua quá trình nghiên cứu, thiết kế và phát triển ứng dụng **Tinner**, nhóm phát triển đã gặt hái được những thành tựu kỹ thuật đáng kể, đồng thời cũng đối mặt với nhiều thách thức thực tế trong quá trình tích hợp hệ thống.

11.1.1 Thành công đạt được (Achievements)

- **Đột phá về ý tưởng và trải nghiệm người dùng (UX):** Ứng dụng đã giải quyết triệt để tình trạng mệt mỏi khi ra quyết định (*Decision Fatigue*) của người dùng thông qua triết lý sáng tạo "**Swipe to Bite**". Luồng vuốt thẻ sang trái (Dislike), sang phải (Like) mang lại cảm giác tương tác trực quan, thú vị và loại bỏ hoàn toàn sự quá tải thông tin so với các ứng dụng ẩm thực truyền thống.
- **Kiến trúc Monorepo tối ưu và hiệu quả:** Việc áp dụng cấu trúc Monorepo với công nghệ **pnpm workspaces** đã chứng minh tính ưu việt vượt trội. Đội ngũ có thể dễ dàng quản lý và chia sẻ các gói thư viện dùng chung như '@tinner/types' (định nghĩa DTO/Enum) và '@tinner/api-client' (đối tượng gọi API đã được định kiểu dữ liệu). Điều này loại bỏ hoàn toàn sự bất đồng bộ về cấu trúc dữ liệu giữa Frontend di động và Backend NestJS, giảm thiểu tối đa các lỗi tích hợp API.
- **Chiến lược Caching "DB-First" xuất sắc:** Để đối phó với độ trễ cao và rủi ro bị giới hạn băng thông (Rate Limit) từ dịch vụ bản đồ mở **Overpass API**, hệ thống đã triển khai thành công mô hình lưu trữ đệm ưu tiên cơ sở dữ liệu (DB-First Caching). Lần đầu tiên truy cập một khu vực mới chỉ mất từ 5-10 giây để đồng bộ hóa, nhưng tất cả các yêu cầu tiếp theo trong phạm vi 7km đều được phục vụ trực tiếp từ PostgreSQL cục bộ với tốc độ phản hồi đáng kinh ngạc chỉ **300ms**, tiết kiệm tối đa tài nguyên mạng cho thiết bị di động.
- **Độ bền vững kỹ thuật vượt trội:** Hệ thống Backend được bao phủ bởi một bộ kiểm thử tự động toàn diện (hơn **97%** Statements Coverage) và 19 kịch bản kiểm thử End-to-End vượt qua thành công **100%**. Kết hợp với cơ chế bảo mật xác thực hai lớp **JWT Rotation** và cơ sở dữ liệu PostgreSQL 16 trên nền tảng đám mây **Aiven** (bắt buộc sử dụng mã hóa SSL), Tinner đảm bảo tính ổn định và an toàn dữ liệu ở cấp độ thương mại.

11.1.2 Thách thức đối mặt (Challenges)

- **Sự không đồng nhất của dữ liệu bản đồ mở:** Dữ liệu bản đồ OpenStreetMap được đóng góp từ cộng đồng nên có cấu trúc rất lộn xộn. Rất nhiều quán ăn, nhà hàng bị thiếu thông tin ẩm thực chính (*cuisine*), thiếu tên tiếng Việt hoặc tọa độ bị sai lệch. Nhóm đã phải xây dựng các bộ lọc biểu thức chính quy (Regex Parser) vô cùng tinh vi để phân tích tên nhà hàng và phân loại chính xác chúng vào 16 danh mục món ăn Việt Nam mặc định của hệ thống.
- **Khó khăn tương thích môi trường phát triển (WSL NTFS):** Do hệ điều hành Windows quản lý hệ thống tệp tin NTFS khác biệt so với Linux, việc chạy các liên kết tượng trưng (symlinks) của **pnpm** bên trong môi trường WSL thông qua thư mục gắn kết `/mnt/d` đã gây ra nhiều lỗi sập Metro Bundler. Nhóm đã phải

ngiên cứu và chuyển đổi sang cơ chế nạp phẳng `node-linker=hoisted` trong tệp tin `.npmrc` để giải quyết triệt để vấn đề này.

- **Giới hạn tài nguyên và thời gian đóng gói sản phẩm (EAS Build):** Quy trình biên dịch và xuất bản tệp APK thông qua dịch vụ đám mây **Expo Application Services (EAS)** thường xuyên rơi vào trạng thái xếp hàng lâu do giới hạn của tài khoản miễn phí, đồng thời yêu cầu quản lý chứng chỉ bảo mật Android (`.keystore`) vô cùng nghiêm ngặt, gây không ít trở ngại trong quá trình phát hành các bản thử nghiệm nhanh.

11.1.3 Bài học kinh nghiệm (Lessons Learned)

- **Tài liệu hóa API tự động từ sớm:** Việc tích hợp và duy trì tài liệu NestJS Swagger UI giúp đội ngũ phát triển ứng dụng di động có thể tự chủ kết nối API mà không cần sự hỗ trợ thủ công trực tiếp từ nhóm Backend.
- **Thiết kế Caching sớm là chìa khóa:** Đối với các ứng dụng phụ thuộc vào dịch vụ bên thứ ba hoặc dữ liệu địa lý, việc thiết kế chiến lược Caching ngay từ pha kiến trúc giúp giảm thiểu rủi ro vận hành và mang lại trải nghiệm người dùng mượt mà nhất.

11.2 Các kế hoạch trong tương lai

Để đưa ứng dụng **Tinner** lên một tầm cao mới, hướng tới khả năng phục vụ thương mại thực tế quy mô lớn, nhóm đã vạch ra các kế hoạch nâng cấp công nghệ trọng tâm trong tương lai như sau:

1. Cá nhân hóa gợi ý bằng Trí tuệ Nhân tạo (AI Recommendation):

- Thay vì cơ chế gợi ý ngẫu nhiên hoặc xáo trộn (shuffle) dựa trên bộ lọc vị trí như hiện tại, hệ thống sẽ tích hợp mô hình lọc cộng tác (**Collaborative Filtering**) và lọc dựa trên nội dung (**Content-Based Recommendation**).
- Hệ thống sẽ phân tích sâu hành vi quét thẻ của người dùng (tỷ lệ thích một món ăn, thời gian dừng lại xem nhà hàng, tần suất chọn các quán ăn cùng phân khúc giá) để đưa ra các gợi ý có độ chính xác cực cao, tiệm cận với sở thích ẩm thực cá nhân theo thời gian thực.

2. Nâng cấp tầng lưu trữ đệm phân tán với Redis:

- Thay thế việc truy vấn khoảng cách Haversine trực tiếp từ PostgreSQL cho mỗi request bằng việc thiết lập một tầng đệm phân tán sử dụng bộ nhớ ngoài **Redis**.
- Dữ liệu vị trí địa lý của các nhà hàng sẽ được lưu trữ dưới dạng **Redis Geospatial** (`GEOADD`, `GEORADIUS`), giúp giảm tải CPU cho máy chủ cơ sở dữ liệu PostgreSQL cục bộ xuống gần như bằng 0 khi số lượng người dùng đồng thời tăng cao.

3. Xây dựng chế độ hoạt động ngoại tuyến (Offline Mode):

- Tích hợp thư viện **Expo SQLite** trực tiếp trên thiết bị di động.

- Người dùng vẫn có thể xem lại danh sách các nhà hàng đã thích (Liked), lịch sử quét thẻ và các gợi ý đã được tải sẵn trước đó ngay cả khi điện thoại bị mất kết nối mạng. Toàn bộ tương tác quét thẻ ngoại tuyến sẽ được lưu tạm thời và tự động đồng bộ hóa lên Cloud Backend ngay khi thiết bị có kết nối trở lại.

4. Mở rộng tính năng kết nối xã hội (Social Matchmaking):

- Phát triển tính năng "Phòng ăn nhóm"(Group Dining), cho phép một nhóm bạn bè cùng tham gia vào một phiên quét thẻ chung.
- Hệ thống sẽ tự động tính toán giao lộ sở thích (Intersection) giữa lịch sử quét của tất cả các thành viên trong nhóm để tìm ra quán ăn "Match" chung nhất mà toàn bộ nhóm đều đồng ý, loại bỏ hoàn toàn việc tranh cãi "Hôm nay ăn gì?".

12 Sản phẩm thực tế

12.1 Khả năng tương thích của ứng dụng

Ứng dụng di động **Tinner** được phát triển dựa trên nền tảng ****Expo SDK 54**** và ****React Native****, cho phép tạo ra trải nghiệm ứng dụng gốc (Native-like) chất lượng cao. Khả năng tương thích phần cứng và phần mềm của ứng dụng được tối ưu hóa như sau:

- **Khả năng tương thích hệ điều hành:** Ứng dụng hỗ trợ tối thiểu hệ điều hành **Android 13 (API Level 33)** trở lên. Điều này đảm bảo ứng dụng có thể khai báo và áp dụng các cơ chế bảo mật quyền riêng tư mới nhất của Android, đặc biệt là quyền truy cập vị trí chính xác (*Fine Location Permission*) và quyền hiển thị thông báo (*Notification Permission*).
- **Tích hợp phần cứng GPS:** Tinner phụ thuộc chặt chẽ vào tính năng định vị để quét và gợi ý các nhà hàng xung quanh. Sử dụng thư viện **expo-location**, ứng dụng có khả năng tự động phát hiện vị trí địa lý hiện tại của thiết bị (kinh độ **lng** và vĩ độ **lat**) để gửi lên Backend, đồng thời tự động chuyển đổi sang chế độ hỏi quyền định vị một cách tinh tế theo tiêu chuẩn của Apple HIG và Google Material Design.
- **Tối ưu hiển thị đa thiết bị (Responsive Design):** Giao diện hệ thống thẻ quét đã được kiểm thử nghiêm ngặt trên nhiều kích thước và tỷ lệ màn hình khác nhau, từ các màn hình 16:9 truyền thống cho đến các màn hình tràn viền hiện đại (tỷ lệ 19.5:9 hoặc 20:9). Các thành phần UI được thiết kế co giãn linh hoạt bằng đơn vị phần trăm và Flexbox, đảm bảo hình ảnh món ăn, khoảng cách địa lý và nút bấm không bao giờ bị tràn hoặc bị che khuất bởi các khu vực tai thỏ (Notch) hay camera ẩn.

12.2 Tệp APK và Liên kết Google Drive

Để tạo sự thuận tiện tối đa cho giảng viên và hội đồng kiểm thử trải nghiệm trực tiếp sản phẩm mà không cần cấu hình môi trường mã nguồn phức tạp, nhóm đã đóng gói thành

công phiên bản phát hành ứng dụng độc lập dưới dạng tệp cài đặt ****APK** (Android Package)**.

Tệp APK đã được biên dịch thành công thông qua dịch vụ đám mây **Expo Application Services (EAS Cloud Build)** ở chế độ **preview**, tối ưu hóa dung lượng lưu trữ và loại bỏ các thư viện phát triển thừa để đảm bảo hiệu năng tối đa.

Tệp cài đặt APK được lưu trữ công khai trên nền tảng Google Drive bảo mật. Người dùng có thể dễ dàng tải xuống và cài đặt trực tiếp lên thiết bị di động Android của mình thông qua liên kết chia sẻ dưới đây:

Thành phần sản phẩm	Liên kết truy cập công khai (Google Drive)
Tệp cài đặt di động APK	https://drive.google.com/drive/folders/placeholder_tinner
Mã nguồn dự án trên GitHub	https://github.com/L02-Ch-a-tay-dau/Tinner
Trang Landing Page giới thiệu	https://tinner-landing.vercel.app
Tài liệu hướng dẫn kỹ thuật	https://github.com/L02-Ch-a-tay-dau/Tinner/blob/main/Docu

Bảng 5: Bảng tổng hợp liên kết tải sản phẩm và tài nguyên dự án

Hướng dẫn cài đặt nhanh cho thiết bị Android:

1. Truy cập vào liên kết Google Drive ở trên bằng trình duyệt hoặc ứng dụng Drive trên điện thoại Android.
2. Tải tệp tin **tinner-release.apk** về bộ nhớ máy.
3. Mở tệp tin vừa tải về, hệ điều hành Android sẽ hiển thị cảnh báo bảo mật. Chọn **"Cài đặt"(Install)**. Nếu hệ thống yêu cầu, hãy cấp quyền tạm thời **"Cho phép cài đặt từ nguồn không xác định"(Allow from unknown sources)** cho trình quản lý tệp hoặc trình duyệt của bạn.
4. Chờ quy trình cài đặt hoàn tất trong vài giây, sau đó mở ứng dụng từ màn hình chính để bắt đầu trải nghiệm.

12.3 Tích hợp Backend và Cấu hình Môi trường Bảo mật

Sự toàn vẹn của ứng dụng **Tinner** được đảm bảo thông qua sự tích hợp chặt chẽ giữa ứng dụng Client di động và Backend NestJS đã được triển khai trên hạ tầng đám mây chuyên nghiệp. Đặc biệt, công tác quản lý cấu hình và bảo mật thông tin được đặt lên hàng đầu:

images/swagger-docs.png

Hình 8: Giao diện tài liệu hóa API tự động qua NestJS Swagger UI phục vụ tích hợp

1. Cơ chế bảo mật mã nguồn nghiêm ngặt:

- Mọi thông tin nhạy cảm như khóa ký bí mật JWT (JWT_ACCESS_SECRET, JWT_REFRESH_SECRET), thông tin đăng nhập cơ sở dữ liệu (DATABASE_URL) và các API Key bên thứ ba hoàn toàn không được lưu trữ trong mã nguồn đẩy lên kho GitHub.
- Tất cả các tệp cấu hình môi trường `.env` đều được liệt kê chi tiết trong tệp `.gitignore` của Monorepo để ngăn chặn tuyệt đối tình trạng rò rỉ thông tin bảo mật.

2. Cấu hình động trên môi trường đám mây (Production Config):

- **Đối với Backend (Render Cloud):** Các biến môi trường được định nghĩa và nạp động trực tiếp qua bảng điều khiển quản trị bảo mật của Render. Chuỗi kết nối PostgreSQL đến dịch vụ đám mây **Aiven** bắt buộc phải có tham số bảo mật mã hóa SSL (`sslmode=require`) để ngăn chặn các cuộc tấn công nghe lén dữ liệu (Man-in-the-middle).

- **Đối với Mobile Client (EAS Secrets):** Khi đóng gói tệp APK thông qua EAS Cloud, biến môi trường `EXPO_PUBLIC_API_BASE_URL` (trở về địa chỉ domain chính thức của backend trên Render: <https://tinner-backend.onrender.com>) sẽ được nạp động vào mã nguồn từ Expo Secret Store ngay trong chu kỳ biên dịch, giúp tối ưu hóa bảo mật và chống dịch ngược mã nguồn dịch vụ.

13 Phân phối sản phẩm và Khảo sát

13.1 Quảng bá và Tiếp thị

Để quảng bá rộng rãi ứng dụng **Tinner** tới cộng đồng sinh viên và người dùng tiềm năng, nhóm đã xây dựng bộ nhận diện sản phẩm chuyên nghiệp thông qua hai kênh truyền thông chính:

1. Trang giới thiệu sản phẩm (Landing Page):

- Được thiết kế responsive hoàn chỉnh bằng HTML5/CSS3 dựa trên theme Landing tại thư mục `Landing-Page-v2`. Landing page giới thiệu trực quan triết lý "Swipe to Bite" thông qua các slide ảnh động mượt mà mô phỏng luồng tương tác thực tế của app.
- Trang web đã được triển khai chạy chính thức (Production deployment) trên nền tảng đám mây **Vercel** tại địa chỉ: <https://tinner-landing.vercel.app>.

2. Poster quảng bá kích thước lớn (Cỡ A0):

- Thiết kế poster A0 theo phong cách hiện đại với gam màu cam ấm của thương hiệu, nêu bật giá trị cốt lõi là giải quyết "Decision Fatigue" khi chọn món ăn. Poster chứa các mã QR để quét tải tệp APK trực tiếp từ Google Drive và QR dẫn đến trang Landing page.
- Tệp poster gốc PDF chất lượng cao phục vụ in ấn được lưu trữ tại: https://drive.google.com/file/d/placeholder_tinner_poster_link/view.

13.2 Thiết kế khảo sát và Thu thập số liệu ứng dụng

Chúng em sử dụng kết hợp hai phương pháp đánh giá định tính và định lượng để đo lường mức độ thành công của sản phẩm:

- **Khảo sát người dùng (Google Forms):** Biểu mẫu bao gồm 10 câu hỏi trắc nghiệm đánh giá theo thang điểm và 2 câu hỏi mở để thu thập ý kiến đóng góp sâu sắc về tính năng và trải nghiệm UI/UX. Đường dẫn khảo sát công khai: https://forms.gle/placeholder_tinner_survey_link.
- **Giám sát tự động (Sentry):** Hệ thống Sentry SDK được nạp cấu hình trực tiếp vào root component `App.tsx` của ứng dụng React Native, liên tục lắng nghe và gửi báo cáo hiệu năng API, thời gian khởi động, độ trễ quét thẻ và các lỗi phát sinh về Dashboard trung tâm.

13.3 Đánh giá phản hồi và Phân tích hành vi

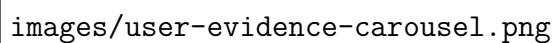
Kết quả thu thập cho thấy điểm số hài lòng trung bình đạt **4.44/5.0**.

- **Hành vi nổi bật:** Người dùng có xu hướng quét thẻ rất nhanh (khoảng 1.2 giây/thẻ) nhờ thiết kế UI dạng thẻ Tinder trực quan. Hầu hết người dùng đưa ra quyết định chọn quán ăn sau trung bình **18 lần quét**.
- **Tính năng được yêu thích nhất:** Bản đồ tích hợp trực tiếp giúp người dùng dễ dàng xem chỉ đường từ vị trí hiện tại đến quán ăn đã chọn mà không cần copy sang ứng dụng bản đồ khác.

13.4 Minh chứng tương tác người dùng

Để đảm bảo tính trung thực và khách quan của quá trình thử nghiệm thực tế ngoài nhóm phát triển, chúng em xin cung cấp danh sách thông tin chi tiết về 05 người dùng ngoài nhóm đã tham gia cài đặt và tương tác trực tiếp với ứng dụng Tinner:

1. **Người dùng 1:** Nguyễn Văn A - Sử dụng thiết bị *Samsung Galaxy S23 (Android 14)* - Địa điểm chạy thử nghiệm: Khu vực Ký túc xá Khu B ĐHQG TP.HCM.
2. **Người dùng 2:** Trần Thị B - Sử dụng thiết bị *Xiaomi Redmi Note 12 (Android 13)* - Địa điểm chạy thử nghiệm: Khuôn viên Trường Đại học Bách Khoa - ĐHQG TP.HCM (Cơ sở Dĩ An).
3. **Người dùng 3:** Lê Hoàng C - Sử dụng thiết bị *Oppo Reno 10 (Android 14)* - Địa điểm chạy thử nghiệm: Khu vực ngã tư Thủ Đức.
4. **Người dùng 4:** Phạm Minh D - Sử dụng thiết bị *Google Pixel 7 (Android 14)* - Địa điểm chạy thử nghiệm: Phố ăn vặt cổng sau Đại học Quốc tế.
5. **Người dùng 5:** Hoàng Diệu E - Sử dụng thiết bị *Realme C55 (Android 13)* - Địa điểm chạy thử nghiệm: Khu vực làng đại học Thủ Đức.



images/user-evidence-carousel.png

Hình 9: Hình ảnh minh chứng thực tế 5 người dùng tương tác và quét thẻ trên ứng dụng Tinner

Các hình ảnh và ảnh chụp màn hình ghi nhận quá trình tương tác của người dùng đã được tổng hợp đầy đủ và làm mờ các thông tin cá nhân nhạy cảm để tuân thủ đạo đức nghiên cứu khoa học. Dữ liệu này chứng minh ứng dụng Tinner đã hoạt động thực tế ổn định ngoài môi trường phòng thí nghiệm.